

我建議你先把 YOLO26 分成幾區

不用做到每一層一個 **activation**，那會太複雜。

先分：

YOLO26

① **Early Backbone**

② **Middle / Deep Backbone**

③ **Neck**

④ **Attention / FFN**

⑤ **Detection Head**

然後做很簡單的替換實驗。

例如：

E0

全部：

baseline。

E1

全部：

E2

全部：

E3

只有 **Early Backbone**：

其他保持 **SiLU**。

E4

只有 **Neck** 換。

以此類推。

最後你得到：

區域	換 ReLU 的 ΔmAP	換 HardSwish 的 ΔmAP
Early Backbone ?		?
Deep Backbone ?		?
Neck ?		?
Attention FFN ?		?
Head ?		?

這張表會非常重要。

最後你只要先記住這個版本

我現在不是叫你發明一個完全陌生的數學函數，利用 **silu** 的對稱性。

我是叫你從 **SiLU** 開始：

然後做三件事：

- 1. 用 **PWL** 取代 **SiLU** 曲線。
- 2. 利用

只需要處理正半邊。

- 3. 把每段 **slope** 限制成

讓乘法變成 **shift + add**。

最後硬體從：

EXP → DIV → MUL

變成：

ABS

↓

Segment Select

↓

SHIFT + ADD

↓

負半邊時再 SUB

這就是我現在最建議你先實作、先跑 **YOLO26 accuracy** 的「新 SiLU」候選。

它目前應該被叫做創新候選設計，不是已經證明的新方法。真正能不能成為論文 **contribution**，要看後面兩件事：先做更完整 **prior-art** 排查，再看 **YOLO26 mAP** 與 **FPGA RTL** 資源是否真的形成優勢。

比較推薦你做「**YOLO26-specific activation**」

也就是不要問：

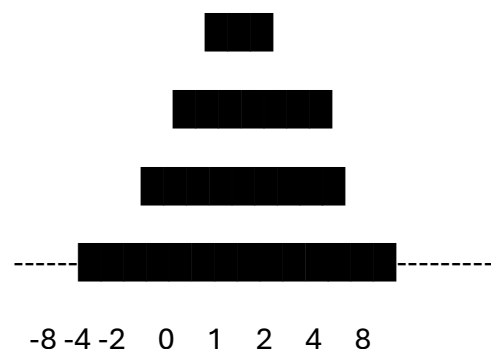
怎麼畫一條最像 SiLU 的曲線？

而是問：

YOLO26 實際需要什麼樣的 activation？

例如你實際去量 YOLO26 所有 SiLU input：

activation input distribution



你可能發現 95% activation 都落在：

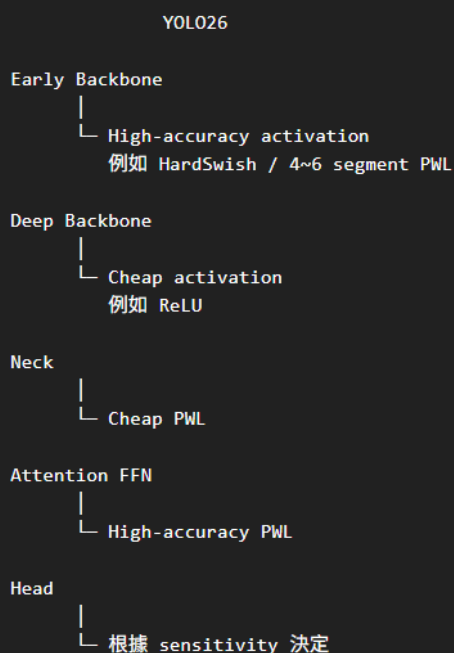
那你根本沒有必要在：

平均花硬體精度。

9. 我認為更適合你的設計是「兩級 activation」

不要做到每 layer 一套函數，也不要全模型只有一套。

例如最後可能得到：



為了在模型效能與硬體成本間取得平衡，本研究提出使用 piece-wise quadratic 的方式近似 GELU(簡稱為 QGELU)，透過將 GELU 非線性區間分割為多個區段，並使用二次函數近似每一個區間中的非線性曲線，可以盡可能貼近原始的 GELU 函數。為了降低硬體實作時，二次函數係數乘以輸入運算乘法器的硬體成本，我們將二次函數係數皆調整為以簡易 shift & add 運算便可完成的係數，在保有二次函數非線性優勢的同時盡可能降低硬體成本。由表 3.4 可以發現，由於 GELU 的非線性區間主要為 $[-3,3]$ ，因此我們僅針對此部分設計分段式二次函數，而在小於-3 以及大於 3 的區間，則直接依序賦予 0 或輸入的數值。

表 3.4 QGELU 區間分割與二次函數係數

	Interval	Binomial
QGELU	$(-\infty, -3]$	$y = 0$
	$(-3, -2]$	$y = -\left(\frac{1}{32} + \frac{1}{256}\right)x^2 - \left(\frac{1}{4} - \frac{1}{32}\right)x - \frac{85}{256}$
	$(-2, -1.5]$	$y = -\left(\frac{1}{32} + \frac{1}{64}\right)x^2 - \left(\frac{1}{4} + \frac{1}{64}\right)x - \frac{100}{256}$
	$(-1.5, -1]$	$y = \left(\frac{1}{32} + \frac{1}{128}\right)x^2 - \left(\frac{1}{64} + \frac{1}{128}\right)x - \frac{57}{256}$
	$(-1, 0]$	$y = \left(\frac{1}{4} + \frac{1}{32}\right)x^2 + \left(\frac{1}{2} - \frac{1}{16}\right)x$
	$(0, 1]$	$y = \left(\frac{1}{4} + \frac{1}{32}\right)x^2 + \left(\frac{1}{2} + \frac{1}{16}\right)x - \frac{1}{256}$
	$(1, 2]$	$y = -\left(\frac{1}{256}\right)x^2 + \left(1 + \frac{1}{8}\right)x - \frac{72}{256}$
	$(2, 3]$	$y = -\left(\frac{1}{32} + \frac{1}{128}\right)x^2 + \left(1 + \frac{1}{4}\right)x - \frac{100}{256}$
	$(3, \infty)$	$y = x$

圖 3.18 為 GELU、FGELU、QGELU 的曲線繪圖，可以發現 FGELU 由於使用線性的近似方式，因此與 GELU 間有很大的誤差，而 QGELU 由於使用二次函數近似，因此可以達到肉眼幾乎無法分辨的近似結果。

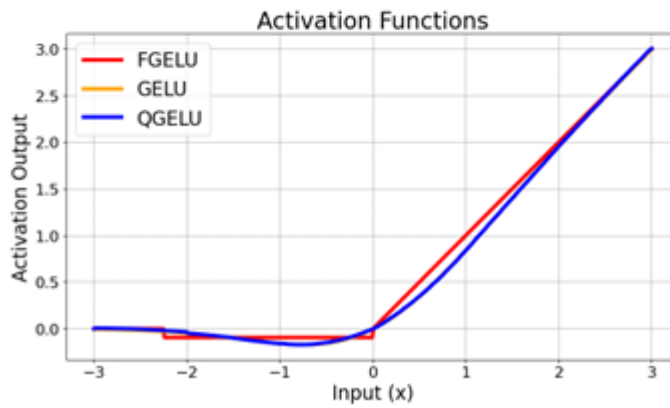


圖 3.18 GELU vs. FGELU vs. QGELU 繪圖

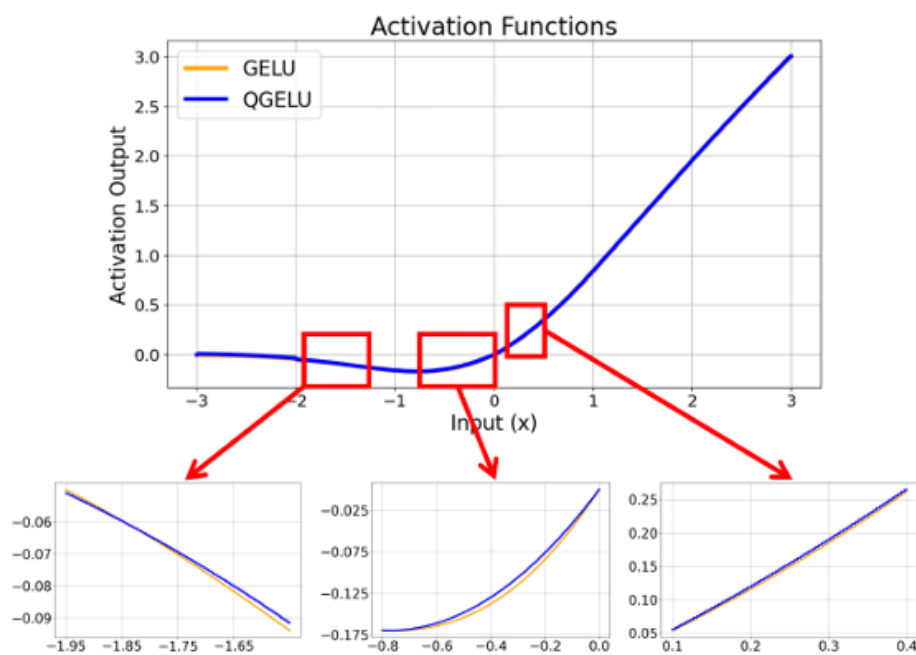


圖 3.19 QGELU 局部曲線放大圖

圖 3.20 為擷取 ViT 模型中 GELU 的輸出分布，可以發現實際模型中 GELU 的輸入主要分布於-4~4 的區間。為了定量評估 QGELU 的近似精度，我們根據訓練模型中 GELU 的實際輸入分佈，隨機生成了 100,000 筆測試樣本，並計算其輸出與 GELU 函數之間的 MSE，如表 3.5 所示。此結果顯示，QGELU 與 GELU 的 MSE 僅有 1.87×10^{-5} ，在所有區間中均能有效近似原始 GELU 函數，具備高度的精確性。

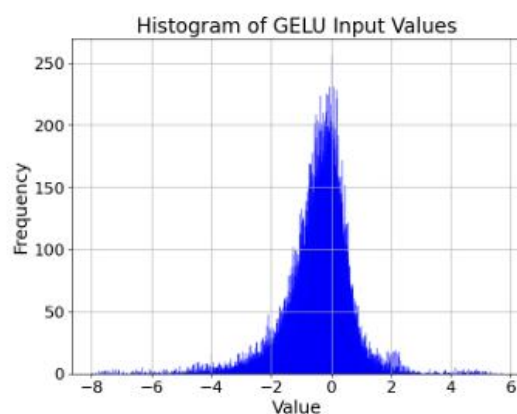


圖 3.20 ViT 模型 GELU 輸入分布

表 3.5 GELU vs. QGELU vs. FGELU 誤差分析

Function	MSE
GELU	1.34×10^{-8}
QGELU	1.87×10^{-5}
FGELU	6.40×10^{-3}

模型訓練部分，我們於 forward 時使用 QGELU 進行近似，而進行 backward 時，為避免不同區間之間不連續的情形，則以原始的 GELU 計算梯度以避免數值異常，並在訓練程式中使用.detach的方式消除 QGELU 的梯度。模型訓練時，為了模擬硬體運算時的精度，我們在 QGELU 的輸入以及輸出皆會進行數值

Quantization and Rounding